Lecture L14

*Blind review — please score this lecture on row **L14** in the Excel form (1–10 per dimension).*

Learning Objectives

By the end of this lecture, you will be able to:

1. **Remember** the purpose of backpropagation and identify its role in training neural networks.
2. **Understand** how the chain rule is applied in reverse order across a computational graph to compute gradients efficiently.
3. **Apply** the backpropagation algorithm to compute gradients in a simple multilayer perceptron (MLP) with one hidden layer.
4. **Analyze** common failure modes of backpropagation (e.g., vanishing gradients) and explain how architectural choices mitigate them.
5. **Create** a mental model of how modern deep learning frameworks automate backpropagation using dynamic computation graphs.

Prerequisites

Before proceeding, ensure familiarity with:

- Basic calculus: derivatives, partial derivatives, and the chain rule.
- Vector and matrix operations, including matrix-vector multiplication and transpose.
- Forward propagation in a neural network: computing activations layer by layer.
- The concept of a loss (or cost) function and its role in training.

If any of these topics are unclear, pause to review them—they are foundational for understanding backpropagation.

Intuition First: Why Do We Need Backpropagation?

Imagine you're debugging a complex Rube Goldberg machine: a chain reaction of levers, balls, and pulleys designed to perform a simple task—say, ringing a bell. After one run, the bell doesn't sound. You know *something* went wrong, but where? The ball missed a lever, which failed to release a weight, which never pulled the string attached to the hammer. Each component contributed, but unequally.

Now suppose the machine has thousands of parts, and you must adjust each one slightly to improve performance. How do you assign blame—or credit—across the entire chain?

This is the **credit assignment problem** in neural networks. Every weight in the network contributes to the final prediction, just like each lever affects whether the bell rings. We need to know: *How much should we tweak each weight to reduce error?* That's where gradients come in.

But here's the catch: computing how a tiny change in an early weight affects the final loss seems to require tracing forward through the entire network—*for every single weight.* For a network with a million parameters, that would mean a million forward passes. Computationally, this is catastrophic.

Backpropagation solves this with a stroke of algorithmic insight: instead of asking, "What happens if I nudge this weight?" a million times, it asks once: "Given the error at the output, how does it *flow backward* through the network to each parameter?" It computes all gradients in just *two* passes: one forward, one backward.

Misconception Alert: Backpropagation is *not* learning. It is a method for computing gradients. Learning occurs when an optimization algorithm, such as gradient descent, uses these gradients to update weights.

Quick Check: Why can't we compute gradients efficiently by finite differences?
(Answer: It would require O(n) forward passes for n parameters—prohibitively expensive. Backpropagation computes all gradients in O(1) backward pass.)

Formal Foundation: The Chain Rule in a Computational Graph

Let's formalize the idea using **computational graphs**—a way to decompose any mathematical expression into nodes (values) and edges (operations). Think of it as a flowchart of computation.

Suppose your loss J depends on a prediction $\hat{y}$, which depends on weights W, which depend on earlier inputs. This dependency forms a directed path. But real networks are not paths—they're **trees**, even **graphs**, with branching and merging.

Here's the key: the gradient of J with respect to any node u_i depends on how u_i influences all downstream nodes. The **multivariable chain rule** gives us the tool:

$$\frac{\partial J}{\partial u_i} = \sum_{j \in \text{children}(u_i)} \frac{\partial J}{\partial u_j} \cdot \frac{\partial u_j}{\partial u_i}$$

This looks recursive—and it is. But instead of computing it top-down (which would repeat work), we compute it *bottom-up*, starting from J and moving backward.

Analogy: Imagine water flowing through a network of pipes from a reservoir (the loss) back to its sources (the parameters). At each junction, the flow splits or merges according to local pipe slopes ($\partial u_j / \partial u_i$). The total water arriving at each source tells you how much that parameter contributed to the final error.

This backward flow is **backpropagation**. And because we reuse each $\partial J / \partial u_j$ when computing its parents, we avoid recomputation. This is **dynamic programming** in disguise: solving a large problem by combining solutions to overlapping subproblems.

Try This Yourself: Draw the computational graph for $J = (w_1 x + w_2)^2$. Label each node. Now compute $\partial J / \partial w_1$ using the chain rule, following the graph backward.
(Hint: Start at J, go to the square, then to the sum, then to $w_1 x$, then to w_1.)

Quick Check: Why can't gradients be computed in forward order?
(Answer: The gradient of the loss depends on downstream nodes; we must start from the output where the loss is defined and move backward.)

Worked Example: Backpropagation in a Simple MLP

We now apply backpropagation to a concrete example: a two-layer multilayer perceptron (MLP) with one hidden layer.

Network Setup The model computes predictions as follows:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{h} = \sigma(\mathbf{z}^{(1)}), \quad \hat{\mathbf{y}} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

We use the mean squared error (MSE) loss:

$$J = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}\|^2$$

The parameters are:

$$\theta = \left\{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}\right\}$$

Our goal is to compute $\nabla_\theta J$, the gradient of J with respect to each parameter.

Step 1: Forward Pass (Review) Given input $\mathbf{x}$, compute and store the following: - $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x}+\mathbf{b}^{(1)}$ - $\mathbf{h} = \sigma(\mathbf{z}^{(1)})$ - $\hat{\mathbf{y}} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$ - $J = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}\|^2$

All intermediate values—$\mathbf{z}^{(1)}, \mathbf{h}, \hat{\mathbf{y}}$—must be stored. They are required during the backward pass for computing local derivatives.

Quick Check: Why do we store $\mathbf{z}^{(1)}$?
(Answer: Because we need $\sigma'(\mathbf{z}^{(1)})$, the derivative of the activation function evaluated at $\mathbf{z}^{(1)}$.)

Step 2: Backward Pass — Output Layer We begin the backward pass at the loss J and propagate gradients backward through the output layer.

Step 2.1: Compute $\delta^{(2)} = \frac{\partial J}{\partial \hat{\mathbf{y}}}$

For mean squared error:

$$\frac{\partial J}{\partial \hat{\mathbf{y}}} = \hat{\mathbf{y}} - \mathbf{y}$$

We denote this vector $\delta^{(2)}$. It represents the error signal at the network output.

Analogy: This is like measuring the final deviation of a missile from its target. You don't yet know which thruster failed, but you know the total miss. This "miss vector" is your starting point for diagnosis.

Step 2.2: Compute gradients for $\mathbf{W}^{(2)}$ and $\mathbf{b}^{(2)}$

Using the chain rule:

$$\frac{\partial J}{\partial \mathbf{W}^{(2)}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{W}^{(2)}} = \delta^{(2)} \cdot \frac{\partial}{\partial \mathbf{W}^{(2)}}(\mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$$

From matrix calculus: - $\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{W}^{(2)}} = \mathbf{h}^\top$ - $\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{b}^{(2)}} = \mathbf{I}$

Thus:

$$\frac{\partial J}{\partial \mathbf{W}^{(2)}} = \delta^{(2)}\mathbf{h}^\top, \quad \frac{\partial J}{\partial \mathbf{b}^{(2)}} = \delta^{(2)}$$

Intuition: The weight gradient is the outer product of output error and hidden activation. Larger errors or stronger activations result in larger gradient magnitudes, leading to bigger updates.

Try This Yourself: Suppose $\delta^{(2)} \in \mathbb{R}^2$, $\mathbf{h} \in \mathbb{R}^3$. What is the shape of $\frac{\partial J}{\partial \mathbf{W}^{(2)}}$? Confirm it matches $\mathbf{W}^{(2)}$.
(Answer: 2×3, same as $\mathbf{W}^{(2)}$.)

Step 3: Backward Pass — Hidden Layer We now propagate the error signal to the hidden layer.

Step 3.1: Compute $\frac{\partial J}{\partial \mathbf{h}}$

Since $\hat{\mathbf{y}} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$, the gradient with respect to $\mathbf{h}$ is:

$$\frac{\partial J}{\partial \mathbf{h}} = \left(\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{h}}\right)^\top \frac{\partial J}{\partial \hat{\mathbf{y}}} = (\mathbf{W}^{(2)})^\top \delta^{(2)}$$

This gives us $\frac{\partial J}{\partial \mathbf{h}} \in \mathbb{R}^3$, a vector of how changes in each hidden unit affect the loss.

Analogy: This is like back-calculating how much each joint in a robotic arm contributed to the end-effector's position error. The motors at the base don't move the hand directly—they act through the chain. So we use the transpose of the weight matrix as a "reverse kinematics" operator.

Step 3.2: Compute $\frac{\partial J}{\partial \mathbf{z}^{(1)}}$

Now, $\mathbf{h} = \sigma(\mathbf{z}^{(1)})$, so:

$$\frac{\partial J}{\partial \mathbf{z}^{(1)}} = \frac{\partial J}{\partial \mathbf{h}} \odot \sigma'(\mathbf{z}^{(1)})$$

We denote this $\delta^{(1)}$. The $\odot$ is element-wise multiplication—the chain rule applied per neuron.

This step is critical: it modulates the upstream gradient by how sensitive the activation function is at $\mathbf{z}^{(1)}$. If a neuron is saturated (e.g., sigmoid near 0 or 1), σ' is near zero, and the gradient vanishes.

> **Quick Check**: What happens to $\sigma'(z)$ when z is very large or very small for a sigmoid activation?
> *(Answer: It approaches 0—this is the root of the vanishing gradient problem.)*

Step 3.3: Compute gradients for $\mathbf{W}^{(1)}$ and $\mathbf{b}^{(1)}$

Now apply the same pattern:

$$\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \delta^{(1)}\mathbf{x}^\top, \quad \frac{\partial J}{\partial \mathbf{b}^{(1)}} = \delta^{(1)}$$

Again, the gradient is the outer product of local error and input.

> **Try This Yourself**: Write the full backward pass as a loop over layers. What changes between layers? What stays the same?
> *(Hint: The pattern is recursive: error in, derivative through activation, propagate backward via weight transpose.)*

Why Backpropagation Works: The Hidden Symmetry

Here's a non-obvious insight most textbooks omit: **backpropagation is the adjoint of forward propagation**.

In forward pass, data flows:
$\mathbf{x} \to \mathbf{W}^{(1)}\mathbf{x} \to \sigma(\cdot) \to \mathbf{W}^{(2)}\mathbf{h}$

In backward pass, error flows:
$\delta^{(2)} \to (\mathbf{W}^{(2)})^\top \delta^{(2)} \to \delta^{(1)} \to (\mathbf{W}^{(1)})^\top \delta^{(1)}$

The weight matrices are transposed—just like in linear algebra, where the adjoint of a linear map reverses the direction of transformation.

Cross-Domain Link: In physics, time-reversible systems have forward and backward dynamics linked by adjoints. Backpropagation is the "time-reversal" of forward propagation, governed by the same matrices but run backward.

This symmetry is why automatic differentiation works: deep learning frameworks record the forward operations and replay them in reverse, transposed.

Failure Modes and Modern Fixes

Backpropagation is elegant—but fragile.

Vanishing Gradients: In deep networks with sigmoid/tanh activations, $\sigma'(z) \ll 1$ in many layers. Multiplying many small numbers causes gradients to shrink exponentially. Early layers learn almost nothing.

Exploding Gradients: Conversely, if gradients are large, they can grow exponentially, causing instability.

Analogy: Imagine whispering a message down a long line of people. If each person speaks softer than the last, the message vanishes. If each speaks louder, it becomes noise.

Architectural Fixes: - **ReLU**: $\sigma'(z) = 1$ for $z > 0$, so gradients pass unchanged. - **Skip Connections (ResNet)**: Create express lanes for gradients to jump across layers. - **Batch Normalization**: Stabilizes z values, keeping $\sigma'(z)$ in a useful range.

> **Quick Check**: Why does ReLU help with vanishing gradients?
> *(Answer: Its derivative is 1 for positive inputs, so gradients don't shrink during backpropagation through ReLU units.)*

Automation: How Frameworks Do It

Modern frameworks (PyTorch, TensorFlow) automate backpropagation using **dynamic computation graphs**.

Every operation (e.g., `+`, `@`, `sigmoid`) records: 1. The inputs, 2. The function applied, 3. A "backward" function that computes local gradients.

During the backward pass, the framework traverses this graph in reverse topological order, applying the chain rule automatically.

Analogy: It's like a spreadsheet that not only computes values but also records *how* each cell was computed. When you ask "How does this total depend on that input?", it traces the formula tree backward.

> **Try This Yourself**: In PyTorch, set `requires_grad=True` on a tensor, perform operations, call `.backward()`, then inspect `.grad`. What do you see?

Summary

Backpropagation is not magic—it's the chain rule, applied efficiently through dynamic programming. It solves the credit assignment problem by propagating error backward, layer by layer, reusing computations.

Its power lies in symmetry: forward pass builds predictions, backward pass builds gradients—using the same weights, but transposed.

And its fragility—vanishing or exploding gradients—has driven innovation in architectures, from ReLU to ResNet.

Now that you've seen the gears inside the engine, you're no longer just running the model—you're ready to fix it when it breaks.